

Predicting Youth Emancipation in Europe Using Socioeconomic and Geographic Indicators

Carlos Seguí Leyton

2026-09-10

1. Loading the Required Libraries

We will start by loading all the necessary libraries.

```
library(tidyverse)
library(eurostat)
library(caret)
library(gam)
library(glmnet)
```

Now, we will load the necessary datasets. Note that we will need to filter and wrangle all of them in order to generate one final dataset and use it to train and test the prediction model. The `eurostat` package gives us data that is already processed, so it should not be difficult to work with it.

2. Loading and wrangling the data

The first database corresponds to the estimated average age of young persons leaving the parental household, coded `yth_demo_030`. The `values` variable will be the output of our model. We will remove the observations corresponding to the global average of the EU, and take the year as for the time period.

```
yth_demo_030 <- get_eurostat("yth_demo_030")
yth_demo_030 <- yth_demo_030 %>%
  filter(!geo %in% c("EU27_2020", "EA21"), sex == "T") %>%
  mutate(TIME_PERIOD = year(TIME_PERIOD),
         age_leaving_home = values) %>%
  select(geo, TIME_PERIOD, age_leaving_home)
```

All the next databases will give us the predictors for our model. The second database corresponds to the Youth unemployment rate by country of birth, coded as `yth_empl_100`.

```
yth_empl_100 <- get_eurostat("yth_empl_100")
yth_empl_100 <- yth_empl_100 %>%
  filter(!geo %in% c("EU27_2020", "EA21"),
         c_birth %in% c("NAT", "EU27_2020_FOR", "NEU27_2020_FOR"),
         sex == "T",
         age == "Y15-29",
         !is.na(values)) %>%
  mutate(TIME_PERIOD = year(TIME_PERIOD),
```

```

      youth_unemp_rate = values) %>%
select(c_birth, geo, TIME_PERIOD, youth_unemp_rate) %>%
pivot_wider(names_from = c_birth, values_from = youth_unemp_rate,
            names_prefix = "unemp_") %>%
filter(!is.na(unemp_NAT),
       !is.na(unemp_EU27_2020_FOR),
       !is.na(unemp_NEU27_2020_FOR))

```

As predictors, we will use the year, age group, country of residence, and country of birth. For the latter variable, we distinguish three categories: native-born (NAT), foreign-born within the EU (EU27_2020_FOR), and foreign-born outside the EU (NEU27_2020_FOR). We restrict the analysis to the Y15-29 age group, the standard definition of “youth” in European statistics. This choice also avoids the overlapping age brackets present in the original data (e.g., Y15-24 and Y20-24), which would otherwise introduce non-independent observations into the model.

The third database corresponds to the mean and median income by age and sex, coded as `ilc_di03`. We will not use the sex variable as predictor.

```

ilc_di03 <- get_eurostat("ilc_di03")
ilc_di03 <- ilc_di03 %>%
  filter(!geo %in% c("EU", "EU15", "EU28", "EA", "EA18", "EA19", "EA20",
                    "EA21", "EU27_2007", "EU27_2020"),
         sex == "T",
         age == "TOTAL",
         unit == "PPS",
         statinfo == "MED_EI",
         !is.na(values)) %>%
  mutate(TIME_PERIOD = year(TIME_PERIOD),
         median_income = values) %>%
  select(geo, TIME_PERIOD, median_income)

```

For the income variable, we use the total population rather than a specific age bracket, since income data by age reflects individual earnings rather than the household context relevant to youth emancipation. We use the median instead of the mean, as median income is a more robust measure of central tendency for skewed income distributions and is less influenced by extreme values. Finally, we use Purchasing Power Standard (PPS) rather than EUR or national currency, since PPS accounts for differences in cost of living across countries, making cross-country comparisons more meaningful.

Now, we will load the fourth database, corresponding to the at-risk-of-poverty rate by poverty threshold, age and sex (EU-SILC and ECHP surveys), coded as `ilc_li02`.

```

ilc_li02 <- get_eurostat("ilc_li02")
ilc_li02 <- ilc_li02 %>%
  filter(!geo %in% c("EU", "EA21", "EU27_2020"),
         sex == "T",
         age == "TOTAL",
         unit == "PC",
         statinfo == "MED_EI",
         rskpovth == "B_60",
         !is.na(values)) %>%
  mutate(TIME_PERIOD = year(TIME_PERIOD),
         at_risk_pov_rate = values) %>%
  select(geo, TIME_PERIOD, at_risk_pov_rate)

```

We use the percentage unit (PC) rather than absolute counts (THS_PER), as a relative measure allows for meaningful comparisons across countries of different population sizes. Regarding the poverty threshold, we select B_60 (“Below 60%”), which corresponds to the standard EU at-risk-of-poverty indicator: the share of the population with an equivalised disposable income below 60% of the national median.

The next dataset is about the housing cost overburden rate by age, sex and poverty status, coded `ilc_lvho07a`.

```
ilc_lvho07a <- get_eurostat("ilc_lvho07a")
ilc_lvho07a <- ilc_lvho07a %>%
  filter(!geo %in% c("EU", "EU15", "EU28", "EA", "EA18", "EA19", "EA20",
                    "EA21", "EU27_2007", "EU27_2020"),
         sex == "T",
         age == "TOTAL",
         rskpovth == "B_60",
         !is.na(values)) %>%
  mutate(TIME_PERIOD = year(TIME_PERIOD),
         housing_cost_overburden = values) %>%
  select(geo, TIME_PERIOD, housing_cost_overburden)
```

Housing cost overburden rate measures the percentage of the population living in households where total housing costs represent more than 40% of disposable income. We include this variable because housing affordability is widely recognized in the literature as one of the strongest predictors of delayed youth emancipation in Europe, particularly in Southern European countries, where high housing costs relative to income are a key barrier to independent living.

The sixth dataset corresponds to the population in private households by educational attainment level, coded `edat_lfse_03`.

```
edat_lfse_03 <- get_eurostat("edat_lfse_03")
edat_lfse_03 <- edat_lfse_03 %>%
  filter(!geo %in% c("EA21", "EU27_2020"),
         sex == "T",
         age == "Y15-29",
         isced11 %in% c("ED0-2", "ED3_4", "ED5-8"),
         !is.na(values)) %>%
  mutate(TIME_PERIOD = year(TIME_PERIOD),
         pop = values) %>%
  select(geo, TIME_PERIOD, isced11, pop) %>%
  pivot_wider(names_from = isced11, values_from = pop,
             names_prefix = "edu_")
```

For this dataset, we retain the three standard ISCED 2011 aggregate levels — low (ED0-2), medium (ED3_4), and high (ED5-8) — as they provide a clear and internationally comparable breakdown of educational attainment, avoiding the redundancy of overlapping categories such as ED3-8 or finer subcategories like general/vocational orientation. We use the Y15-29 age group for consistency with the youth definition applied throughout the analysis, ensuring that all youth-related indicators refer to the same population.

At this point, you might have noticed that, for the youth unemployment (`yth_empl_100`) and educational attainment (`edat_lfse_03`) datasets, we reshape the data from long to wide format using `pivot_wider()`, converting each category (country of birth and ISCED education level, respectively) into a separate column. This keeps one row per country-year, as needed for the predictive model, and avoids creating unwanted combinations of categories when merging with the other datasets.

The penultimate dataset corresponds to the GDP per capita measure, and is coded `nama_10_pc`.

```

nama_10_pc <- get_eurostat("nama_10_pc")
nama_10_pc <- nama_10_pc %>%
  filter(!geo %in% c("EA", "EA12", "EA19", "EA20", "EA21", "EU27_2020"),
         na_item == "B1GQ",
         unit == "CP_PPS_EU27_2020_HAB",
         !is.na(values)) %>%
  mutate(TIME_PERIOD = year(TIME_PERIOD),
         gdp_per_capita = values) %>%
  select(geo, TIME_PERIOD, gdp_per_capita)

```

We select current prices, purchasing power standard (CP_PPS_EU27_2020_HAB) as the `unit`, consistent with the PPS-based measures used elsewhere in the analysis, allowing meaningful comparisons across countries. For `na_item`, we use B1GQ (Gross domestic product at market prices), the standard and most widely used measure of GDP, rather than its individual components.

The last dataset that we will use is the Young persons neither in employment nor in education and training by labour status (NEET rates), coded `edat_lfse_20`.

```

edat_lfse_20 <- get_eurostat("edat_lfse_20")
edat_lfse_20 <- edat_lfse_20 %>%
  filter(!geo %in% c("EA21", "EU27_2020"),
         sex == "T",
         age == "Y15-29",
         wstatus == "NEMP",
         !is.na(values)) %>%
  mutate(TIME_PERIOD = year(TIME_PERIOD),
         neet_rate = values) %>%
  select(geo, TIME_PERIOD, neet_rate)

```

Here, we use the `wstatus` value NEMP (“Not employed persons”), which represents the overall NEET rate. The other categories (UNE, INAC, WANT, NWANT) are subgroups that break down this total by employment status, so using NEMP avoids redundancy with these more granular breakdowns.

2.1. Building the final dataset

At this point, we will do the adequate joins to build the final dataset.

```

final_df <- yth_demo_030 %>%
  left_join(edat_lfse_03, by = c("geo", "TIME_PERIOD")) %>%
  left_join(edat_lfse_20, by = c("geo", "TIME_PERIOD")) %>%
  left_join(ilc_di03, by = c("geo", "TIME_PERIOD")) %>%
  left_join(ilc_li02, by = c("geo", "TIME_PERIOD")) %>%
  left_join(ilc_lvho07a, by = c("geo", "TIME_PERIOD")) %>%
  left_join(nama_10_pc, by = c("geo", "TIME_PERIOD")) %>%
  left_join(yth_empl_100, by = c("geo", "TIME_PERIOD"))

df_inner <- yth_demo_030 %>%
  inner_join(edat_lfse_03, by = c("geo", "TIME_PERIOD")) %>%
  inner_join(edat_lfse_20, by = c("geo", "TIME_PERIOD")) %>%
  inner_join(ilc_di03, by = c("geo", "TIME_PERIOD")) %>%
  inner_join(ilc_li02, by = c("geo", "TIME_PERIOD")) %>%
  inner_join(ilc_lvho07a, by = c("geo", "TIME_PERIOD")) %>%

```

```
inner_join(nama_10_pc, by = c("geo", "TIME_PERIOD")) %>%
inner_join(yth_empl_100, by = c("geo", "TIME_PERIOD"))

setdiff(unique(final_df$geo), unique(df_inner$geo))
```

```
## [1] "BA" "BG" "EE" "HU" "LT" "LV" "ME" "NO" "PL" "RO" "SI" "SK"
```

We use `left_join()`, starting from the youth emancipation age dataset, to combine all data sources. This approach preserves every country-year observation with a valid target value, while an `inner_join()` would have retained only the intersection of all eight datasets. Testing confirmed that `inner_join` disproportionately excluded Eastern European and non-EU countries (e.g., Bulgaria, Poland, Romania, Norway), introducing systematic geographic bias. The `left_join()` avoids this by keeping the full set of countries and allowing missing values to be assessed and handled explicitly, rather than being silently dropped.

```
final_df %>%
  filter(geo %in% c("BA", "BG", "EE", "HU", "LT", "LV", "ME", "NO", "PL", "RO", "SI", "SK")) %>%
  group_by(geo) %>%
  summarise(across(where(is.numeric), ~ sum(is.na(.)))) %>%
  pivot_longer(-geo, names_to = "variable", values_to = "n_na") %>%
  filter(n_na > 0) %>%
  group_by(variable) %>%
  summarise(n_countries_affected = n(), total_na = sum(n_na)) %>%
  arrange(desc(total_na))
```

```
## # A tibble: 7 x 3
##   variable                n_countries_affected total_na
##   <chr>                    <int>         <int>
## 1 unemp_EU27_2020_FOR      12           235
## 2 unemp_NAT                12           235
## 3 unemp_NEU27_2020_FOR    12           235
## 4 housing_cost_overburden  11           37
## 5 median_income           11           36
## 6 at_risk_pov_rate         11           29
## 7 gdp_per_capita           1            1
```

```
final_df_no_birth <- final_df %>%
  select(-unemp_NAT, -unemp_EU27_2020_FOR, -unemp_NEU27_2020_FOR)
```

We detect that the vast majority of NA's come from the country-of-birth variable. Therefore, the country-of-birth unemployment breakdown (`unemp_NAT`, `unemp_EU27_2020_FOR`, `unemp_NEU27_2020_FOR`) was excluded from the final dataset. Missing value analysis showed 235 NA values in each of these columns, concentrated entirely in 12 countries, mainly Eastern European and non-EU states. This indicates the country-of-birth breakdown is not available for these countries in Eurostat, rather than isolated missing years. Retaining these variables would have reintroduced the same geographic bias that `left_join` was intended to avoid, so they were dropped in favor of a more geographically balanced dataset.

3. Making predictions with different models

3.1. Generalized Linear Model, GLM

We will start by making a linear regression model to predict the values of `age_leaving_home`. First, we set the seed to 1, and divide the data into a test set and a train set. We will use the 85% of the data to train the algorithms while reserving the other 15% for testing them.

At this point, we want to make sure that every country has, at least, one observation in the test set and one in the training set. Since our data has a panel structure (multiple years per country) and socioeconomic indicators tend to remain fairly stable within a country over time, a random row-wise split (e.g. using `createDataPartition()`) could place different years of the same country in both the train and test sets. This would allow the model to indirectly learn country-specific patterns during training, inflating test performance and giving an overly optimistic estimate of how well the model generalizes to unseen countries. To avoid this, we instead split the data by country, ensuring that all observations from a given country appear exclusively in either the train or the test set. So, for making the test and train sets, we do the following:

```
set.seed(1)

# 1. We take a representative value for every country (i.e.: mean)
country_avg <- final_df_no_birth %>%
  group_by(geo) %>%
  summarise(avg_target = mean(age_leaving_home, na.rm = TRUE))

# 2. We create groups/strata with that value
country_avg <- country_avg %>%
  mutate(stratum = ntile(avg_target, 4))

# 3. We show countries inside every group/stratum
test_countries <- country_avg %>%
  group_by(stratum) %>%
  slice_sample(prop = 0.15) %>%
  pull(geo)

test_set <- final_df_no_birth %>% filter(geo %in% test_countries)
train_set <- final_df_no_birth %>% filter(!geo %in% test_countries)
```

Now, we will check that both set means for the outcome variable are similar:

```
mean(train_set$age_leaving_home, na.rm = TRUE)
```

```
## [1] 26.99345
```

```
mean(test_set$age_leaving_home, na.rm = TRUE)
```

```
## [1] 26.48767
```

So, the prediction accuracy with a generalized linear model is the one below. We will use the `train()` function from the `caret` package. Also, we exclude `geo` from the set of predictors before fitting the model. Since we split the data by country to test generalization to unseen countries, including `geo` as a categorical predictor would be problematic: the model would learn country-specific effects that cannot be applied to countries not present in the training set, causing prediction to fail entirely for new countries. Removing `geo` forces the model to rely solely on the socioeconomic indicators, in line with the goal of learning generalizable relationships rather than memorizing country identity.

```
# we need to remove NA values to fit the model
train_set <- train_set %>% drop_na() %>% select(-geo)
test_set <- test_set %>% drop_na() %>% select(-geo)

set.seed(1)
```

```
train_glm <- train(age_leaving_home ~ ., method = "glm", data = train_set)
pred_glm <- predict(train_glm, test_set)
glm_error <- RMSE(pred_glm, test_set$age_leaving_home)
```

3.2. Locally Estimated Scatterplot Smoothing Model, LOESS

We will use the same structure to make a prediction with the Loess model:

```
set.seed(1)
train_loess <- train(age_leaving_home ~ .,
                     method = "gamLoess", data = train_set)
```

```
## Warning: model fit failed for Resample01: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample02: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample03: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample04: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample05: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample06: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample07: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample08: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample09: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample10: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample11: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample12: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample13: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample14: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample15: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample16: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample17: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample18: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
```

```
## Warning: model fit failed for Resample19: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample20: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample21: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample22: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample23: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample24: span=0.5, degree=1 Error in eval(predvars, data, env) : ob
## Warning: model fit failed for Resample25: span=0.5, degree=1 Error in eval(predvars, data, env) : ob

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

## Something is wrong; all the RMSE metric values are missing:
##      RMSE      Rsquared      MAE
## Min.   : NA   Min.   : NA   Min.   : NA
## 1st Qu.: NA   1st Qu.: NA   1st Qu.: NA
## Median : NA   Median : NA   Median : NA
## Mean   :NaN   Mean   :NaN   Mean   :NaN
## 3rd Qu.: NA   3rd Qu.: NA   3rd Qu.: NA
## Max.   : NA   Max.   : NA   Max.   : NA
## NA's   :1     NA's   :1     NA's   :1
```

```
## Error: Stopping
```

```
# pred_loess <- predict(train_loess, test_set)
# loess_error <- RMSE(pred_loess, test_set$age_leaving_home)
```

We see that the local regression model (gamLoess) failed to converge during cross-validation, likely due to the relatively high number of predictors combined with a limited sample size — a known limitation of loess-based methods in higher-dimensional settings. This model will therefore be excluded from the final comparison.

3.3. Regularized Regression Model (Elastic Net), glmnet

We will also fit a regularized regression model (Elastic Net, via *glmnet*), which combines ridge and lasso penalties to address potential multicollinearity among predictors. Given that we suspect some multicollinearity among the predictors (income, poverty, and housing—which are likely correlated with one another), *glmnet* is a natural extension of the GLM; it penalizes large coefficients and can better handle that collinearity.

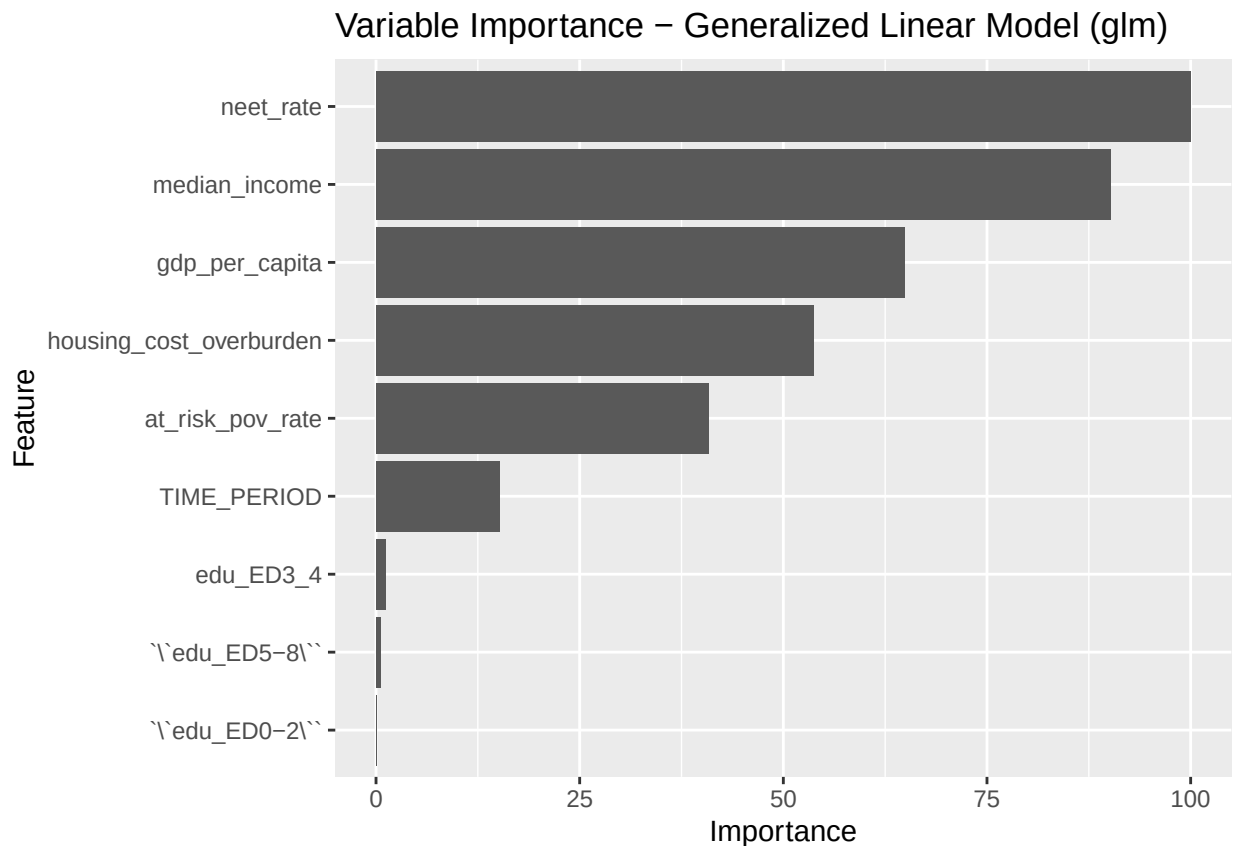
```
set.seed(1)
train_en <- train(age_leaving_home ~ ., method = "glmnet", data = train_set)
pred_en <- predict(train_en, test_set)
en_error <- RMSE(pred_en, test_set$age_leaving_home)
```

At this point, we want to see the variable importance in the linear models:

```
varImp(train_glm)
```

```
## glm variable importance
##
## Overall
## neet_rate 100.0000
## median_income 90.1337
## gdp_per_capita 64.8411
## housing_cost_overburden 53.7211
## at_risk_pov_rate 40.8361
## TIME_PERIOD 15.1910
## edu_ED3_4 1.1722
## '\\edu_ED5-8\\' 0.5465
## '\\edu_ED0-2\\' 0.0000
```

```
ggplot(varImp(train_glm)) +
  labs(title = "Variable Importance - Generalized Linear Model (glm)")
```

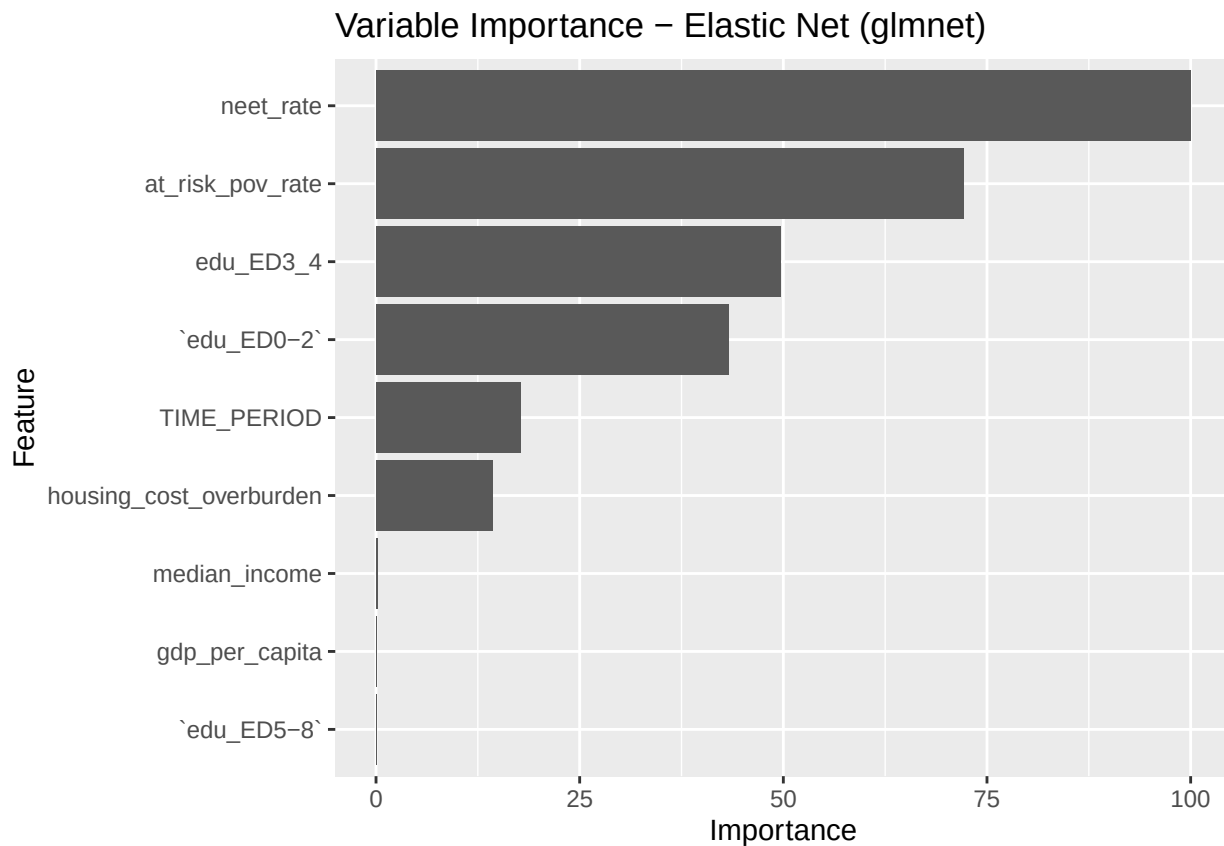


```
varImp(train_en)
```

```
## glmnet variable importance
##
## Overall
## neet_rate 100.00000
```

```
## at_risk_pov_rate      72.17346
## edu_ED3_4            49.70090
## `edu_ED0-2`          43.32147
## TIME_PERIOD          17.68547
## housing_cost_overburden 14.31386
## median_income         0.12505
## gdp_per_capita        0.03476
## `edu_ED5-8`          0.00000
```

```
ggplot(varImp(train_en)) +
  labs(title = "Variable Importance - Elastic Net (glmnet)")
```



Both linear models (glm and glmnet) identify **neet_rate** as by far the most important predictor, with median income, GDP per capita and the housing cost overburden rate also ranking highly in glm, while glmnet instead emphasizes at risk poverty rate and the education variables. Notably, the two models disagree substantially on the role of income and education: glm gives income-related variables high importance and education variables near-zero importance, whereas glmnet does the opposite. This divergence is likely explained by multicollinearity among predictors (e.g., income, GDP and poverty risk are closely related), which can make unregularized coefficient-based importance unstable; glmnet's penalty term redistributes importance differently across correlated variables, which explains why the two linear methods, despite similar overall RMSE, produce different importance rankings.

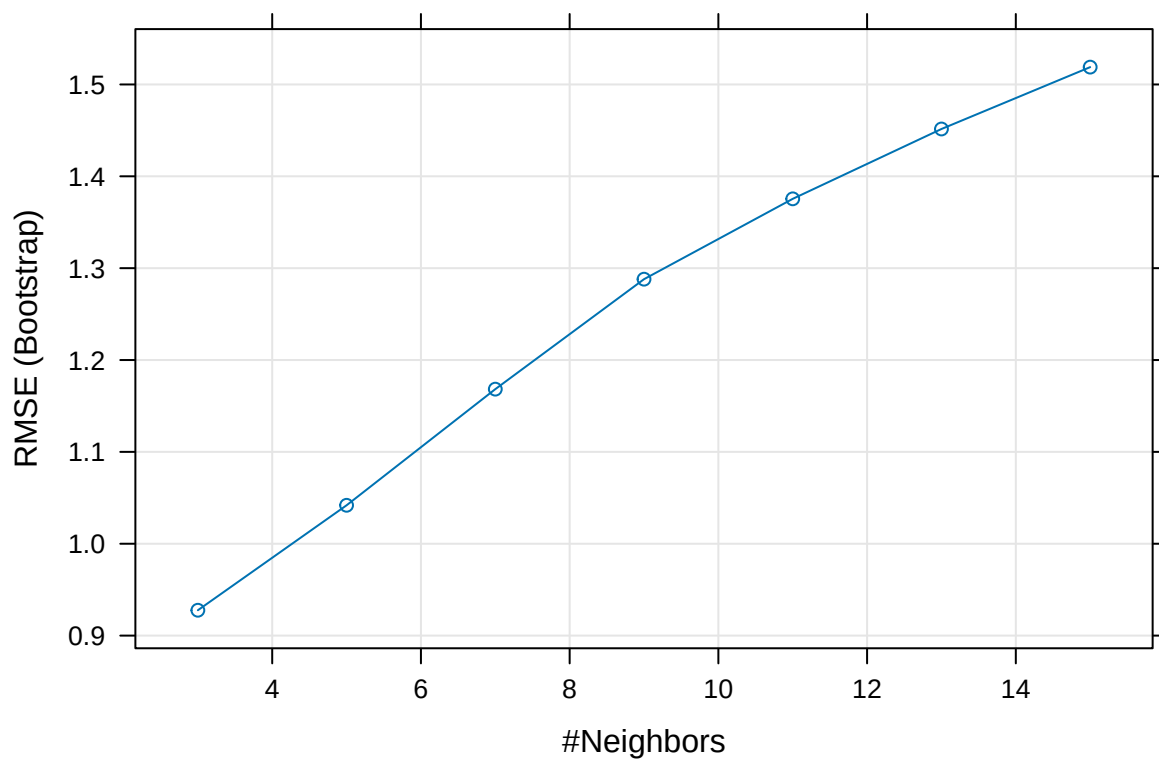
3.4. K-Nearest Neighbors Model, kNN

We will use now the k-nearest neighbors model:

```
set.seed(1)
train_knn <- train(age_leaving_home ~ ., method = "knn",
  data = train_set,
  preProcess = c("center", "scale"),
  tuneGrid = data.frame(k = seq(3, 15, 2)))
train_knn$finalModel
```

```
## 3-nearest neighbor regression model
```

```
plot(train_knn)
```



```
pred_knn <- predict(train_knn, test_set)
knn_error <- RMSE(pred_knn, test_set$age_leaving_home)
```

Until now, it is the model with the highest RMSE; therefore, the less precise model.

3.5. Random Forests, RF

Let's see how random forests are able to predict the outcome. In this case, we will also see which predictors are more important.

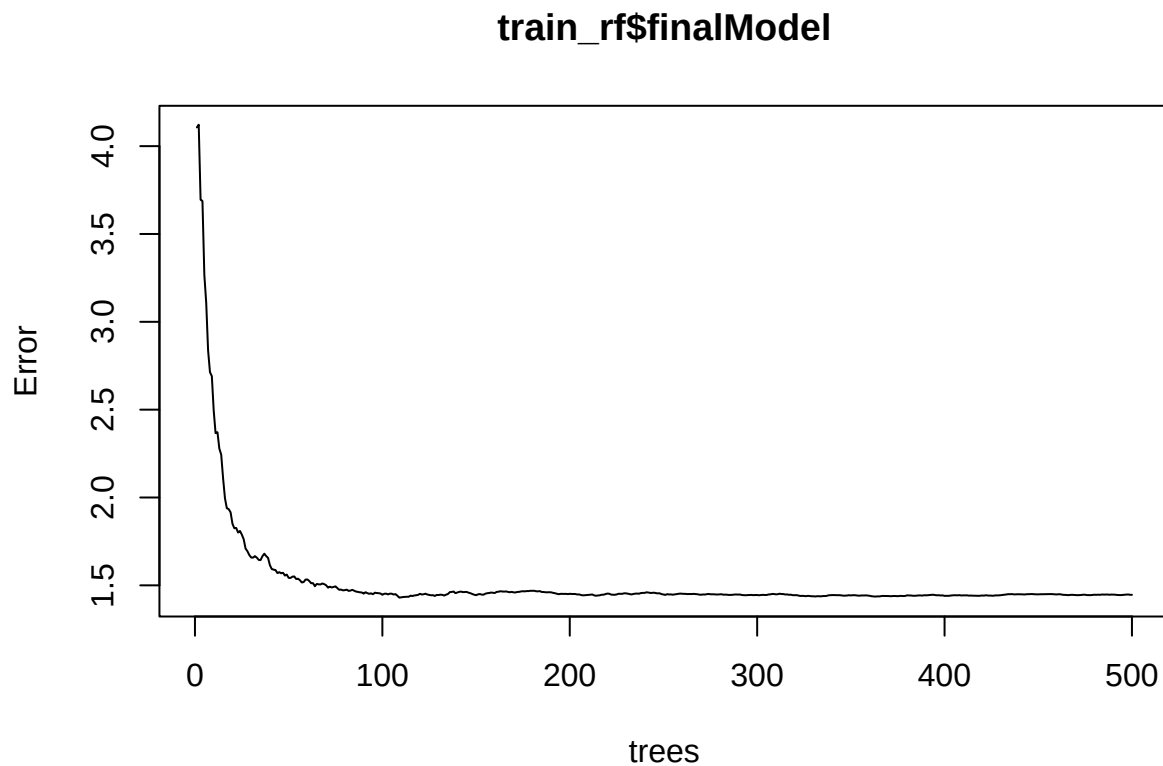
```

set.seed(1)
train_rf <- train(age_leaving_home ~ ., method = "rf",
                  data = train_set,
                  importance = TRUE)
train_rf$finalModel

##
## Call:
##  randomForest(x = x, y = y, mtry = param$mtry, importance = TRUE)
##               Type of random forest: regression
##               Number of trees: 500
## No. of variables tried at each split: 2
##
##               Mean of squared residuals: 1.446124
##               % Var explained: 84.86

plot(train_rf$finalModel)

```



```

pred_rf <- predict(train_rf, test_set)
rf_error <- RMSE(pred_rf, test_set$age_leaving_home)
varImp(train_rf)

```

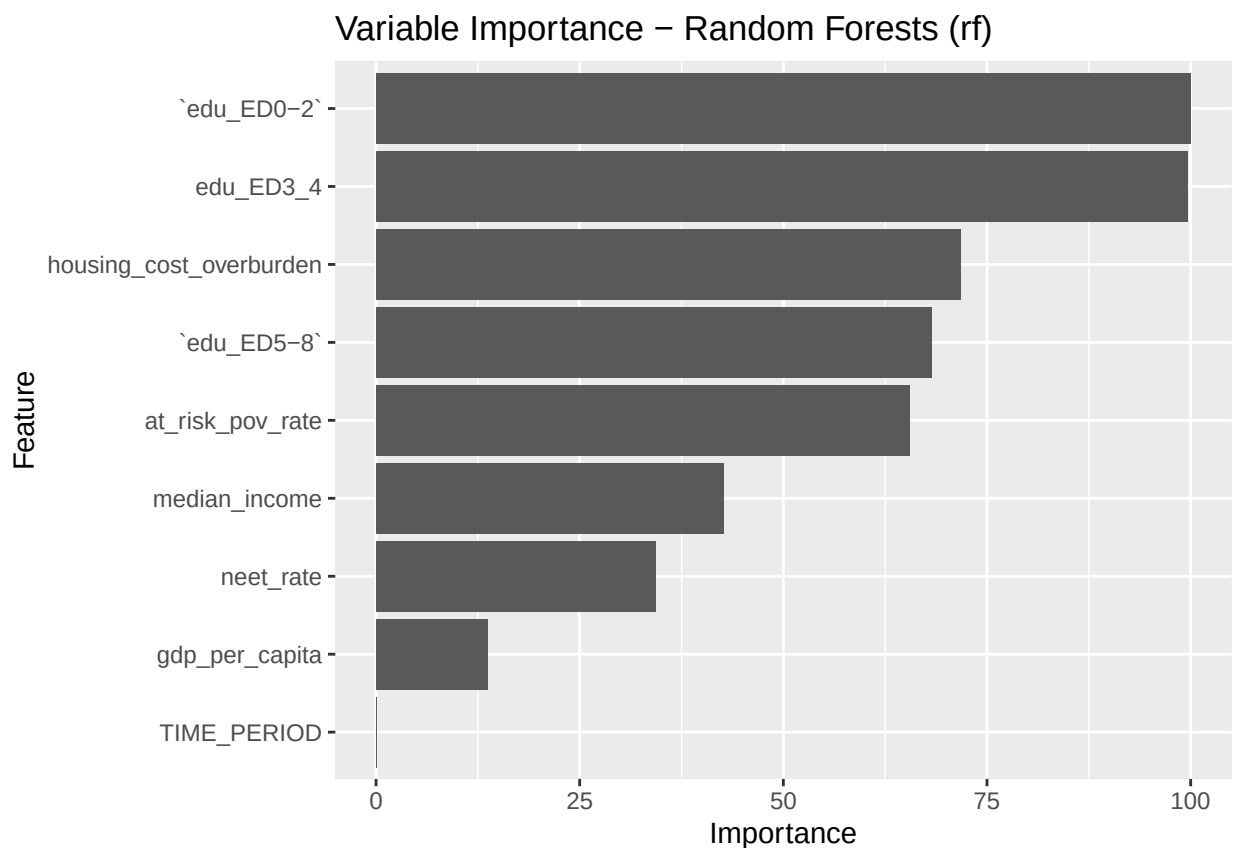
```

## rf variable importance
##

```

```
##                                Overall
## 'edu_ED0-2'                    100.00
## edu_ED3_4                      99.65
## housing_cost_overburden        71.72
## 'edu_ED5-8'                    68.16
## at_risk_pov_rate               65.48
## median_income                  42.62
## neet_rate                      34.32
## gdp_per_capita                 13.68
## TIME_PERIOD                    0.00
```

```
ggplot(varImp(train_rf)) +
  labs(title = "Variable Importance - Random Forests (rf)")
```



We see that is more precise than the kNN method but less that the other two. Regarding the variable importance analysis, random forest paints a markedly different picture when comparing it with the linear models. It shows the education variables and the housing cost overburden rate emerging as the strongest predictors, while neet_rate, the top predictor in both linear models, drops to one of the lowest positions along with the GDP per capita. This discrepancy reflects a fundamental difference in how importance is computed: linear models rank variables by the size and significance of their coefficients, while random forest measures how much each variable actually improves prediction accuracy across many decision splits, capturing non-linear effects and interactions that linear models cannot. This makes random forest's importance ranking generally more reliable when predictors are correlated, and suggests that educational attainment and housing costs may play a more central role in shaping youth emancipation age than the linear models alone would indicate, potentially through non-linear effects or interactions that these models cannot capture.

3.6. Combining all the previous working methods, Ensemble

We will do the mean of all the predictions from the working methods, and then calculate the RMSE.

```
pred_ensemble <- (pred_glm + pred_en + pred_knn + pred_rf)/4
ensemble_error <- RMSE(pred_ensemble, test_set$age_leaving_home)
```

We can also combine the linear methods, as they are the ones with the lowest RMSE:

```
pred_linear <- (pred_glm + pred_en)/2
ensemble_linear_error <- RMSE(pred_linear, test_set$age_leaving_home)
```

The general combination does not improve the best method, and the linear one provides a very similar error to the GLM model.

4. Conclusion

Let's see the errors of the different methods:

```
model_results <- tibble(
  Model = c("Linear regression (glm)",
            "Regularized regression (glmnet)",
            "K-Nearest Neighbors (knn)",
            "Random Forest (rf)",
            "Ensemble (all models)",
            "Ensemble (linear models only)"),
  RMSE = c(glm_error,
            en_error,
            knn_error,
            rf_error,
            ensemble_error,
            ensemble_linear_error)
) %>%
  arrange(RMSE)

knitr::kable(model_results, digits = 4, caption = "Model comparison by RMSE")
```

Table 1: Model comparison by RMSE

Model	RMSE
Linear regression (glm)	2.2752
Ensemble (linear models only)	2.2753
Regularized regression (glmnet)	2.2760
Ensemble (all models)	2.4299
Random Forest (rf)	2.7021
K-Nearest Neighbors (knn)	2.9719

This project set out to predict the average age at which young people leave their parental home across European countries, using a range of socioeconomic indicators: youth unemployment, educational attainment, median income, poverty risk, housing cost burden, GDP per capita, and NEET rates. Data from eight

Eurostat datasets were cleaned, harmonized, and combined into a single country-year panel, with particular attention paid to methodological choices such as unit selection, age group consistency, and handling of missing values, several of which revealed real patterns in data availability across European regions.

A key methodological decision was to split the data by country rather than randomly by row, ensuring that no country appeared in both the training and test sets. This choice was made to test whether the models could generalize to countries they had never seen, rather than simply learning country-specific patterns from repeated observations over time. As a consequence, the country variable itself was excluded from the models, forcing predictions to rely solely on the underlying socioeconomic indicators.

Five modeling approaches were compared: linear regression, regularized linear regression (elastic net), k-nearest neighbors, random forest, and an ensemble combining multiple models. A local regression approach (loess) was also attempted but failed to converge, most likely due to the relatively high number of predictors relative to the sample size, a known limitation of that method.

The results show that the linear models performed best, with linear regression achieving the lowest error, closely followed by the ensemble of linear models and regularized regression — essentially indistinguishable from one another. More flexible models performed worse: the full ensemble combining all four models did not outperform the linear models, and random forest and k-nearest neighbors showed the highest errors. This suggests that the relationship between the socioeconomic indicators used and youth emancipation age is largely linear and additive, without strong non-linear effects or complex interactions that more flexible models could exploit. It also indicates that, once housing costs, income, and educational attainment are accounted for, adding model complexity does not necessarily translate into better predictions, particularly with a moderately sized dataset.

Overall, the model estimates the emancipation age within roughly two years of its true value, on average, for countries not seen during training. Given the complexity and diversity of the factors that shape young people’s transition to independent living, this level of accuracy suggests that the selected socioeconomic indicators, particularly educational attainment and housing affordability, capture a meaningful part of the story, even though country-specific policies, cultural norms, and other unobserved factors are very likely to explain some of the remaining variation.